

# BobaLink Browser Extension API Reference

**Document Version:** 1.0.0  
**Last Updated:** 2026-06-06  
**Classification:** Public API Developer Reference  
**Companion Document:** [Technical Specification](#) | [Developer Guide](#)

---

## Table of Contents

- 1. [Overview](#)
  - 2. [Internal Extension APIs](#)
  - 3. [Boba API Integration](#)
  - 4. [qBitTorrent WebUI API Reference](#)
  - 5. [TypeScript Type Definitions](#)
  - 6. [Error Codes and Handling](#)
  - 7. [Rate Limiting](#)
  - 8. [Version Compatibility Matrix](#)
- 

## 1. Overview

This document provides exhaustive API reference material for three categories of interfaces:

- 1. **Internal Extension APIs:** The message-passing protocol between content scripts, service worker, popup, and options page.
- 2. **Boba API Integration:** RESTful endpoints exposed by the Boba orchestration server, including authentication, search, download, and SSE streaming.
- 3. **qBitTorrent WebUI API:** Complete endpoint documentation for direct qBitTorrent integration, including all parameters and curl/JS fetch examples.

All code examples are provided in **curl**, **JavaScript (fetch)**, and **TypeScript** where applicable. Request and response schemas are specified in TypeScript interface notation.

### Base URLs

| Environment     | Boba Base URL           | qBitTorrent Base URL     |
|-----------------|-------------------------|--------------------------|
| Local (default) | https://boba.local:8443 | https://localhost:8080   |
| Docker          | https://boba:8443       | https://qbittorrent:8080 |
| Custom          | User-configured         | User-configured          |

### Authentication Summary

| API       | Auth Method | Header / Mechanism |
|-----------|-------------|--------------------|
| Boba REST | API Key     |                    |

| API               | Auth Method | Header / Mechanism                                    |
|-------------------|-------------|-------------------------------------------------------|
|                   |             | X-API-Key: {key} or<br>Authorization: Bearer<br>{jwt} |
| Boba SSE          | API Key     | X-API-Key: {key}                                      |
| qBitTorrent       | Cookie      | Automatic SID cookie after /<br>auth/login            |
| qBitTorrent (alt) | Basic Auth  | Authorization: Basic<br>{base64}                      |

---

## 2. Internal Extension APIs

The extension uses `chrome.runtime.sendMessage()` for communication between scripts. All messages follow a uniform envelope format.

### 2.1 Message Envelope

```
interface ExtensionMessage<T = unknown> {
  /** Message type discriminator */
  type: MessageType;

  /** Request correlation ID for async response matching */
  requestId?: string;

  /** Message payload â€” type depends on `type` */
  payload: T;

  /** Sender context (auto-populated by runtime) */
  sender?: {
    tabId?: number;
    frameId?: number;
    url?: string;
  };
}

type MessageType =
  | 'SCAN_REQUEST'
  | 'SCAN_RESPONSE'
  | 'TORRENT_DETECTED'
  | 'SEND_TORRENT'
  | 'SEND_BATCH'
  | 'GET_STATUS'
  | 'STATUS_RESPONSE'
  | 'GET_QUEUE'
  | 'QUEUE_RESPONSE'
  | 'RETRY_ITEM'
  | 'REMOVE_ITEM'
  | 'CLEAR_QUEUE'
  | 'GET_CONFIG'
  | 'CONFIG_RESPONSE'
  | 'UPDATE_CONFIG'
```

```
| 'CONFIG_UPDATED'
| 'GET_HEALTH'
| 'HEALTH_RESPONSE'
| 'DISCOVER_SERVERS'
| 'DISCOVERY_RESPONSE'
| 'TEST_CONNECTION'
| 'CONNECTION_TEST_RESULT';
```

## 2.2 SCAN\_REQUEST

Sent from the service worker (or popup) to a content script to trigger a page scan.

**Direction:** Service Worker → Content Script

**Delivery:** `chrome.tabs.sendMessage(tabId, message)`

```
interface ScanRequestPayload {
  /** Unique request identifier for correlation */
  requestId: string;

  /** When true, scan invisible/hidden links as well */
  deepScan: boolean;

  /** Optional CSS selector to scope the scan */
  scope?: string;
}
```

**Example → Sending from Service Worker:**

```
// TypeScript
const requestId = crypto.randomUUID();
const response = await chrome.tabs.sendMessage(tabId, {
  type: 'SCAN_REQUEST',
  requestId,
  payload: { requestId, deepScan: false }
});

// JavaScript
const requestId = crypto.randomUUID();
chrome.tabs.sendMessage(tabId, {
  type: 'SCAN_REQUEST',
  requestId,
  payload: { requestId, deepScan: false }
}, (response) => {
  if (chrome.runtime.lastError) {
    console.error('Scan failed:', chrome.runtime.lastError.message);
    return;
  }
  console.log('Found torrents:', response.payload.torrents);
});
```

## 2.3 SCAN\_RESPONSE

Sent from the content script back to the service worker with scan results.

**Direction:** Content Script → Service Worker

**Response to:** SCAN\_REQUEST

```
interface ScanResponsePayload {
  /** Correlation ID matching the SCAN_REQUEST */
  requestId: string;

  /** Detected torrents (deduplicated by content script) */
  torrents: TorrentInfo[];

  /** Error message if scan failed */
  error?: string;

  /** Metadata about the scan */
  meta: {
    linksScanned: number;
    magnetsFound: number;
    torrentFilesFound: number;
    scanDurationMs: number;
  };
}
```

## 2.4 TORRENT\_DETECTED

Sent proactively by the content script when magnet links or torrent file links are detected during normal browsing (via MutationObserver).

**Direction:** Content Script → Service Worker

**Delivery:** `chrome.runtime.sendMessage(message)`

```
interface TorrentDetectedPayload {
  /** Array of newly detected torrents */
  torrents: TorrentInfo[];

  /** URL of the page where detection occurred */
  pageUrl: string;

  /** Unix timestamp (ms) of detection */
  timestamp: number;
}
```

### Example “Content Script Detection:

```
const magnetLinks = document.querySelectorAll('a[href^="magnet:"]');
const torrents = Array.from(magnetLinks).map(parseMagnetElement);

if (torrents.length > 0) {
  chrome.runtime.sendMessage({
    type: 'TORRENT_DETECTED',
    payload: {
      torrents,
      pageUrl: location.href,
      timestamp: Date.now()
    }
  });
}
```

```
    });  
}
```

## 2.5 SEND\_TORRENT

Queue or immediately send a single torrent to the configured download server.

**Direction:** Popup/Context Menu â†’ Service Worker

```
interface SendTorrentPayload {  
    /** The torrent to send */  
    torrent: TorrentInfo;  
  
    /** Send immediately even if queue has items */  
    priority?: 'normal' | 'high';  
  
    /** Override default category for this send */  
    category?: string;  
  
    /** Override default tags for this send */  
    tags?: string[];  
}
```

**Example:**

```
chrome.runtime.sendMessage({  
    type: 'SEND_TORRENT',  
    payload: {  
        torrent: detectedTorrent,  
        priority: 'high',  
        category: 'movies'  
    }  
});
```

## 2.6 SEND\_BATCH

Send multiple torrents as a single batch operation.

**Direction:** Tab Manager â†’ Service Worker

```
interface SendBatchPayload {  
    /** Array of torrents to send */  
    torrents: TorrentInfo[];  
  
    /** Optional category override for the entire batch */  
    category?: string;  
  
    /** Source tab group ID, for logging */  
    tabGroupId?: number;  
}
```

**Response:**

```

interface SendBatchResponse {
  /** Number of torrents successfully queued/sent */
  accepted: number;

  /** Number of torrents rejected (e.g., duplicates) */
  rejected: number;

  /** Number of torrents that failed immediately */
  failed: number;

  /** Detailed results per torrent */
  results: Array<{
    infohash: string;
    status: 'queued' | 'sent' | 'duplicate' | 'failed';
    error?: string;
  }>;
}

```

## 2.7 GET\_STATUS

Request the current connection and queue status.

**Direction:** Popup → Service Worker

**Response:** STATUS\_RESPONSE

// Request has empty payload  
 interface GetStatusPayload {}

```

interface StatusResponsePayload {
  /** Server connection state */
  connection: 'connected' | 'connecting' | 'disconnected' | 'error';

  /** Server URL being used */
  serverUrl: string;

  /** Server version, if connected */
  serverVersion?: string;

  /** Number of items in the offline queue */
  queueSize: number;

  /** Number of items currently retrying */
  pendingCount: number;

  /** Number of items in dead-letter state */
  deadLetterCount: number;

  /** Last successful health check timestamp */
  lastHealthCheck?: number;

  /** Extension version */
  extensionVersion: string;
}

```

## 2.8 Queue Management Messages

### GET\_QUEUE

```
interface GetQueuePayload {
  /** Filter by status; omit for all */
  filter?: 'pending' | 'retrying' | 'failed' | 'dead-letter';

  /** Maximum items to return (default 100) */
  limit?: number;

  /** Offset for pagination */
  offset?: number;
}

interface QueueResponsePayload {
  items: QueueItem[];
  total: number;
  filtered: number;
}
```

### RETRY\_ITEM

```
interface RetryItemPayload {
  /** Queue item ID to retry */
  itemId: string;
}
```

### REMOVE\_ITEM

```
interface RemoveItemPayload {
  /** Queue item ID to remove */
  itemId: string;
}
```

### CLEAR\_QUEUE

```
interface ClearQueuePayload {
  /** Clear only items matching this status; omit for all */
  status?: 'pending' | 'retrying' | 'failed' | 'dead-letter';
}
```

## 2.9 Configuration Messages

### GET\_CONFIG

```
// Request: empty payload
// Response: full ExtensionConfig object (see Type Definitions section)
```

## UPDATE\_CONFIG

```
interface UpdateConfigPayload {  
  /** Partial config update â€” only provided fields are merged */  
  config: DeepPartial<ExtensionConfig>;  
}
```

The service worker validates all fields, applies the merge, and persists to encrypted storage. A CONFIG\_UPDATED event is broadcast to all listening contexts.

## 2.10 Discovery Messages

### DISCOVER\_SERVERS

```
interface DiscoverServersPayload {  
  /** Probe timeout in ms (default 5000) */  
  timeout?: number;  
  
  /** Specific IP addresses to probe; uses defaults if omitted */  
  targets?: string[];  
}
```

### DISCOVERY\_RESPONSE

```
interface DiscoveryResponsePayload {  
  /** Discovered server endpoints */  
  servers: Array<{  
    url: string;  
    name: string;  
    version: string;  
    responseTimeMs: number;  
    healthy: boolean;  
  }>;  
  
  /** Number of targets probed */  
  targetsProbed: number;  
  
  /** Scan duration in ms */  
  durationMs: number;  
}
```

---

## 3. Boba API Integration

The Boba server acts as a proxy and orchestration layer between the extension and qBitTorrent. It provides additional capabilities including search aggregation, metadata enrichment, and SSE-based progress streaming.



### 3.1 Authentication

#### POST /api/v1/auth/token

Exchange an API key for a short-lived JWT token.

##### Request:

```
interface AuthTokenRequest {  
  /** Boba API key (from server admin) */  
  apiKey: string;  
}
```

##### curl:

```
curl -X POST https://boba.local:8443/api/v1/auth/token \  
  -H "Content-Type: application/json" \  
  -d '{"apiKey": "bob_live_abc123xyz"}'
```

##### JavaScript (fetch):

```
const response = await fetch('https://boba.local:8443/api/v1/auth/token',  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ apiKey: 'bob_live_abc123xyz' })  
);  
const data = await response.json();  
// data.token â€” JWT for subsequent requests  
// data.expiresAt â€” Unix timestamp of expiry
```

##### Response (200 OK):

```
{  
  "token": "eyJhbGciOiJIUzI1NiIs...",  
  "expiresAt": 1735689600,  
  "type": "Bearer"  
}
```

##### Response (401 Unauthorized):

```
{  
  "error": "E_AUTH_INVALID_KEY",  
  "message": "The provided API key is invalid or revoked."  
}
```

#### POST /api/v1/auth/refresh

Refresh an expiring JWT token.

##### curl:

```
curl -X POST https://boba.local:8443/api/v1/auth/refresh \  
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..."
```

### Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...new...",
  "expiresAt": 1735776000
}
```

## 3.2 Health Check

### GET /api/v1/health

Check Boba server availability and version. No authentication required.

#### curl:

```
curl https://boba.local:8443/api/v1/health
```

#### JavaScript:

```
const response = await fetch('https://boba.local:8443/api/v1/health');
const data = await response.json();
```

### Response (200 OK):

```
{
  "status": "healthy",
  "version": "1.4.2",
  "uptime": 86400,
  "extensions": {
    "qbittorrent": "connected",
    "search": "available",
    "metadata": "available"
  }
}
```

## 3.3 Search API

### GET /api/v1/torrents/search

Search for torrents across configured indexers.

#### Query Parameters:

| Parameter | Type    | Required | Description                                  |
|-----------|---------|----------|----------------------------------------------|
| q         | string  | Yes      | Search query string                          |
| category  | string  | No       | Filter by category (e.g., movies, tv, music) |
| limit     | integer | No       | Max results (default 50, max 100)            |
| offset    | integer | No       | Pagination offset                            |

#### curl:

```
curl "https://boba.local:8443/api/v1/torrents/search?q=inception+2010&category=movies"
-H "X-API-Key: bob_live_abc123xyz"
```

### JavaScript:

```
const params = new URLSearchParams({
  q: 'inception 2010',
  category: 'movies',
  limit: '10'
});
const response = await fetch(
  `https://boba.local:8443/api/v1/torrents/search?${params}`,
  { headers: { 'X-API-Key': 'bob_live_abc123xyz' } }
);
const data = await response.json();
```

### Response (200 OK):

```
{
  "results": [
    {
      "infohash": "a1b2c3d4e5f6789012345678901234567890abcd",
      "displayName": "Inception (2010) 1080p BluRay",
      "magnetUri": "magnet:?xt=urn:btih:a1b2c3d4e5f6789012345678901234567890abcd",
      "size": 2147483648,
      "seeders": 150,
      "leechers": 25,
      "source": "1337x",
      "category": "movies",
      "uploadedAt": "2026-01-15T08:30:00Z"
    }
  ],
  "total": 47,
  "query": "inception 2010",
  "categories": ["movies", "tv"]
}
```

## 3.4 Download API

### POST /api/v1/torrents/download

Submit a torrent for download via Boba.

### Request Body:

```
interface DownloadRequest {
  /** Magnet URI or infohash */
  magnetUri?: string;
  infohash?: string;

  /** Category for qBitTorrent */
  category?: string;

  /** Tags for qBitTorrent */
  tags?: string[];
```

```

tags?: string[];

/** Override download path */
savePath?: string;

/** Rename the torrent */
rename?: string;

/** Pause after adding */
paused?: boolean;

/** Skip hash check */
skipChecking?: boolean;

/** Sequential download */
sequentialDownload?: boolean;

/** First/last piece priority */
firstLastPiecePrio?: boolean;
}

```

#### **curl:**

```

curl -X POST https://boba.local:8443/api/v1/torrents/download \
-H "Content-Type: application/json" \
-H "X-API-Key: bob_live_abc123xyz" \
-d '{
  "infohash": "a1b2c3d4e5f6789012345678901234567890abcd",
  "category": "movies",
  "tags": ["1080p", "blu-ray"],
  "sequentialDownload": false
}'

```

#### **JavaScript:**

```

const response = await fetch('https://boba.local:8443/api/v1/torrents/download', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-API-Key': 'bob_live_abc123xyz'
  },
  body: JSON.stringify({
    infohash: 'a1b2c3d4e5f6789012345678901234567890abcd',
    category: 'movies',
    tags: ['1080p', 'blu-ray']
  })
});
const data = await response.json();

```

#### **Response (202 Accepted):**

```

{
  "jobId": "job_abc123",
  "status": "queued",

```

```
    "infohash": "a1b2c3d4e5f6789012345678901234567890abcd",
    "estimatedStart": "2026-06-06T12:05:00Z"
}
```

#### Response (409 Conflict â€” duplicate):

```
{
  "error": "E_DUPLICATE",
  "message": "Torrent already exists in download queue",
  "existingInfohash": "a1b2c3d4e5f6789012345678901234567890abcd"
}
```

### 3.5 SSE Streaming

#### GET /api/v1/events/download-progress

Real-time download progress via Server-Sent Events.

##### curl:

```
curl "https://boba.local:8443/api/v1/events/download-progress" \
  -H "X-API-Key: bob_live_abc123xyz" \
  -H "Accept: text/event-stream" \
  --no-buffer
```

##### JavaScript (EventSource):

```
const evtSource = new EventSource(
  'https://boba.local:8443/api/v1/events/download-progress?apiKey=bob_live'
);

evtSource.addEventListener('progress', (event) => {
  const data = JSON.parse(event.data);
  console.log(`Torrent ${data.infohash}: ${(data.progress * 100).toFixed(1)}%`);
  console.log(`Speed: ${formatBytes(data.speed)}/s, ETA: ${data.eta}s`);
});

evtSource.addEventListener('completed', (event) => {
  const data = JSON.parse(event.data);
  console.log(`Torrent completed: ${data.infohash}`);
});

evtSource.addEventListener('error', (event) => {
  console.error('SSE connection error');
  // Auto-reconnect handled by EventSource
});
```

##### Event Types:

| Event    | Data Fields                                       | Description              |
|----------|---------------------------------------------------|--------------------------|
| progress | infohash, progress, speed, eta, downloaded, total | Periodic progress update |

| Event     | Data Fields                     | Description             |
|-----------|---------------------------------|-------------------------|
| completed | infohash, completionTime, ratio | Download finished       |
| error     | infohash, error, code           | Download error          |
| stopped   | infohash, reason                | Download paused/stopped |

#### Progress Event Schema:

```
interface ProgressEvent {
  infohash: string;
  progress: number;      // 0.0 to 1.0
  speed: number;         // Bytes per second
  eta: number;           // Estimated seconds remaining
  downloaded: number;    // Bytes downloaded
  total: number;         // Total bytes
  numSeeds: number;
  numLeechs: number;
  state: 'downloading' | 'stalledDL' | 'checkingDL' | 'metaDL';
}
```

---

## 4. qBitTorrent WebUI API Reference

This section documents the qBitTorrent WebUI API endpoints used by BobaLink. All endpoints are relative to the configured qBitTorrent base URL (e.g., `https://localhost:8080`).

### 4.1 Authentication

#### POST /api/v2/auth/login

Authenticate and establish a session.

##### Request:

| Parameter | Type   | Required | Description    |
|-----------|--------|----------|----------------|
| username  | string | Yes      | WebUI username |
| password  | string | Yes      | WebUI password |

##### curl:

```
curl -X POST "https://localhost:8080/api/v2/auth/login" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Referer: https://localhost:8080" \
  -d "username=admin" \
  -d "password=adminadmin" \
  -c cookies.txt
```

##### JavaScript:

```
const params = new URLSearchParams();
params.append('username', 'admin');
params.append('password', 'adminadmin');
```

```
const response = await fetch('https://localhost:8080/api/v2/auth/login', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
    'Referer': 'https://localhost:8080'
  },
  body: params,
  credentials: 'include' // Important: accept and send cookies
});

if (response.status === 200) {
  const body = await response.text();
  // body === "Ok.SID" on success
  console.log('Login successful, SID cookie set');
} else {
  console.error('Login failed');
}
```

**Response:** - **200 OK** with body `Ok.SID` "Success, SID cookie set." - **403 Forbidden** "Invalid credentials." - **429 Too Many Requests** "Too many failed login attempts."

**Important Notes:** - The `Referer` header **MUST** match the qBitTorrent origin or the request will be rejected. - The SID cookie is automatically managed by the browser's cookie store. - Call `/api/v2/auth/logout` to terminate the session.

### POST /api/v2/auth/logout

Terminate the current session.

**curl:**

```
curl -X POST "https://localhost:8080/api/v2/auth/logout" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

**Response:** 200 OK (always succeeds, even if session was already expired).

## 4.2 Torrent Management

### POST /api/v2/torrents/add

Add one or more torrents (magnet URIs or .torrent files).

**Request Parameters:**

| Parameter | Type   | Required | Description                                  |
|-----------|--------|----------|----------------------------------------------|
| urls      | string | No*      | Magnet URIs or HTTP links, newline-separated |
| torrents  | file   | No*      | .torrent file(s) as multipart upload         |
| savepath  | string | No       |                                              |

| Parameter          | Type                | Required | Description                          |
|--------------------|---------------------|----------|--------------------------------------|
|                    |                     |          | Download directory path              |
| cookie             | string              | No       | Cookie to send with download request |
| category           | string              | No       | Category to assign                   |
| tags               | string              | No       | Comma-separated tags                 |
| skip_checking      | string (true/false) | No       | Skip hash check                      |
| paused             | string (true/false) | No       | Add paused                           |
| root_folder        | string (true/false) | No       | Create root folder                   |
| rename             | string              | No       | Rename torrent                       |
| upLimit            | integer             | No       | Upload limit (bytes/s)               |
| dLLimit            | integer             | No       | Download limit (bytes/s)             |
| ratioLimit         | float               | No       | Share ratio limit                    |
| seedingTimeLimit   | integer             | No       | Seeding time limit (minutes)         |
| autoTMM            | string (true/false) | No       | Automatic Torrent Management         |
| sequentialDownload | string (true/false) | No       | Sequential download                  |
| firstLastPiecePrio | string (true/false) | No       | First/last piece priority            |

\* At least one of urls or torrents must be provided.

#### curl â€” Magnet URI:

```
curl -X POST "https://localhost:8080/api/v2/torrents/add" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -F "urls=magnet:?xt=urn:btih:a1b2c3d4e5f6789012345678901234567890abcd&dn=Example" \
  -F "category=movies" \
  -F "tags=hd,2026" \
  -F "paused=false"
```

#### curl â€” .torrent File:

```
curl -X POST "https://localhost:8080/api/v2/torrents/add" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -F "torrents=@/path/to/file.torrent" \
  -F "category=software" \
  -F "sequentialDownload=true"
```

#### JavaScript â€” Magnet URI:

```
const formData = new FormData();
formData.append('urls',
  'magnet:?xt=urn:btih:a1b2c3d4e5f6789012345678901234567890abcd&dn=Example'
);
formData.append('category', 'movies');
formData.append('tags', 'hd,2026');
```



```
const response = await fetch('https://localhost:8080/api/v2/torrents/add',
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: formData,
  credentials: 'include'
});

if (response.status === 200) {
  console.log('Torrent added successfully');
} else if (response.status === 415) {
  console.error('Torrent file type not supported');
} else {
  console.error('Add failed:', response.status);
}
```

### JavaScript “ .torrent File:

```
const fileInput = document.getElementById('torrent-file');
const file = fileInput.files[0];

const formData = new FormData();
formData.append('torrents', file, file.name);
formData.append('category', 'software');

const response = await fetch('https://localhost:8080/api/v2/torrents/add',
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: formData,
  credentials: 'include'
});
```

**Response Codes:** | Status | Meaning | “” | 200 | Success “ torrent(s) added | | 400 | Bad Request “ missing urls/torrents | | 403 | Not authenticated | | 415 | Unsupported Media Type | | 500 | Internal Server Error |

### GET /api/v2/torrents/info

List all torrents with optional filtering.

#### Query Parameters:

| Parameter | Type   | Description                                                                                                                       |
|-----------|--------|-----------------------------------------------------------------------------------------------------------------------------------|
| filter    | string | all, downloading, seeding, completed, paused, active, inactive, resumed, stalled, stalled_uploading, stalled_downloading, errored |
| category  | string | Filter by category                                                                                                                |
| tag       | string | Filter by tag                                                                                                                     |

| Parameter | Type    | Description                                  |
|-----------|---------|----------------------------------------------|
| sort      | string  | Sort column (e.g., name, progress, added_on) |
| reverse   | boolean | Reverse sort order                           |
| limit     | integer | Max results                                  |
| offset    | integer | Pagination offset                            |
| hashes    | string  | Pipe-separated infohashes                    |

#### curl:

# All downloading torrents

```
curl "https://localhost:8080/api/v2/torrents/info?filter=downloading&limit=50"
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

# Specific torrent by hash

```
curl "https://localhost:8080/api/v2/torrents/info?hashes=a1b2c3d4e5f6789012345678901234567890"
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

#### JavaScript:

```
const params = new URLSearchParams({
  filter: 'downloading',
  sort: 'progress',
  reverse: 'true',
  limit: '50'
});

const response = await fetch(
  `https://localhost:8080/api/v2/torrents/info?${params}`,
  {
    headers: { 'Referer': 'https://localhost:8080' },
    credentials: 'include'
  }
);

const torrents = await response.json();

for (const t of torrents) {
  console.log(`${t.name}: ${((t.progress * 100).toFixed(1))}%`);
  console.log(`  Speed: DL ${t.dlspeed} / UL ${t.upspeed}`);
  console.log(`  State: ${t.state}, ETA: ${t.eta}`);
}
```

#### Response (Array of Torrent objects):

```
[
  {
    "hash": "a1b2c3d4e5f6789012345678901234567890abcd",
    "name": "Ubuntu 24.04 LTS Desktop",
    "magnet_uri": "magnet:?xt=urn:btih:a1b2c3d4...",
    "size": 6270988288,
    "progress": 0.7534,
```

```

    "dlspeed": 5242880,
    "upspeed": 1048576,
    "priority": 1,
    "num_seeds": 150,
    "num_complete": 2500,
    "num_leechs": 25,
    "num_incomplete": 50,
    "ratio": 0.125,
    "eta": 1800,
    "state": "downloading",
    "seq_dl": false,
    "f_l_piece_prio": false,
    "category": "linux",
    "tags": "iso,official",
    "super_seeding": false,
    "force_start": false,
    "save_path": "/downloads/linux",
    "added_on": 1717776000,
    "completion_on": 0,
    "tracker": "https://torrent.ubuntu.com/announce",
    "dl_limit": 0,
    "up_limit": 0,
    "downloaded": 4724464025,
    "uploaded": 590558003,
    "downloaded_session": 1048576000,
    "uploaded_session": 209715200,
    "amount_left": 1546524263,
    "completed": 0,
    "max_ratio": -1,
    "max_seeding_time": -1,
    "seen_complete": 1717779600,
    "last_activity": 1717783200,
    "total_size": 6270988288,
    "time_active": 7200,
    "seeding_time": 0,
    "seeding_time_limit": -1,
    "availability": 1.85,
    "reannounce": 0,
    "stalled": false
  }
]

```

## POST /api/v2/torrents/delete

Delete one or more torrents.

### Parameters:

| Parameter   | Type                | Required | Description                       |
|-------------|---------------------|----------|-----------------------------------|
| hashes      | string              | Yes      | Pipe-separated infohashes, or all |
| deleteFiles | string (true/false) | No       | Also delete downloaded files      |

**curl:**

```
curl -X POST "https://localhost:8080/api/v2/torrents/delete" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "hashes=a1b2c3d4e5f6789012345678901234567890abcd" \
  -d "deleteFiles=false"
```

**JavaScript:**

```
const params = new URLSearchParams();
params.append('hashes', 'a1b2c3d4e5f6789012345678901234567890abcd');
params.append('deleteFiles', 'false');

await fetch('https://localhost:8080/api/v2/torrents/delete', {
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: params,
  credentials: 'include'
});
```

**Response:** 200 OK (no body).

**POST /api/v2/torrents/pause**

Pause torrent(s).

**Parameters:****Parameter Type Required Description**

|        |        |     |                                   |
|--------|--------|-----|-----------------------------------|
| hashes | string | Yes | Pipe-separated infohashes, or all |
|--------|--------|-----|-----------------------------------|

**curl:**

```
curl -X POST "https://localhost:8080/api/v2/torrents/pause" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "hashes=a1b2c3d4e5f6789012345678901234567890abcd"
```

**JavaScript:**

```
await fetch('https://localhost:8080/api/v2/torrents/pause', {
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: new URLSearchParams({
    hashes: 'a1b2c3d4e5f6789012345678901234567890abcd'
  }),
  credentials: 'include'
});
```

**POST /api/v2/torrents/start**

Resume paused torrent(s).

### Parameters:

| Parameter | Type   | Required | Description                       |
|-----------|--------|----------|-----------------------------------|
| hashes    | string | Yes      | Pipe-separated infohashes, or all |

**curl:**

```
curl -X POST "https://localhost:8080/api/v2/torrents/start" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "hashes=a1b2c3d4e5f6789012345678901234567890abcd|b2c3d4e5f67890123456
```

## POST /api/v2/torrents/recheck

Force recheck of torrent(s).

### Parameters:

| Parameter | Type   | Required | Description               |
|-----------|--------|----------|---------------------------|
| hashes    | string | Yes      | Pipe-separated infohashes |

**curl:**

```
curl -X POST "https://localhost:8080/api/v2/torrents/recheck" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "hashes=a1b2c3d4e5f6789012345678901234567890abcd"
```

## POST /api/v2/torrents/reannounce

Force reannounce to tracker(s).

### Parameters:

| Parameter | Type   | Required | Description               |
|-----------|--------|----------|---------------------------|
| hashes    | string | Yes      | Pipe-separated infohashes |

### 4.3 Categories and Tags

## GET /api/v2/torrents/categories

List all categories.

**curl:**

```
curl "https://localhost:8080/api/v2/torrents/categories" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

## JavaScript:

```
const response = await fetch(
  'https://localhost:8080/api/v2/torrents/categories',
  {
    headers: { 'Referer': 'https://localhost:8080' },
    credentials: 'include'
  }
);
const categories = await response.json();
// Returns: { "movies": { "name": "movies", "savePath": "/downloads/movies"
```

## POST /api/v2/torrents/createCategory

Create a new category.

| Parameter | Type   | Required | Description       |
|-----------|--------|----------|-------------------|
| category  | string | Yes      | Category name     |
| savePath  | string | No       | Default save path |

### curl:

```
curl -X POST "https://localhost:8080/api/v2/torrents/createCategory" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "category=4k-movies" \
  -d "savePath=/downloads/movies/4k"
```

## GET /api/v2/torrents/tags

List all tags.

### curl:

```
curl "https://localhost:8080/api/v2/torrents/tags" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

**Response:** ["hd", "1080p", "bluray", "2026"]

## POST /api/v2/torrents/addTags

Add tags to torrent(s).

| Parameter | Type   | Required | Description                       |
|-----------|--------|----------|-----------------------------------|
| hashes    | string | Yes      | Pipe-separated infohashes, or all |
| tags      | string | Yes      | Comma-separated tags              |

### curl:

```
curl -X POST "https://localhost:8080/api/v2/torrents/addTags" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
```

```
-d "hashes=a1b2c3d4e5f6789012345678901234567890abcd" \  
-d "tags=important,seeding"
```

## 4.4 Synchronization

### GET /api/v2/sync/maindata

Get incremental sync data (all torrents, categories, tags, server state).

| Parameter | Type    | Required | Description                                                 |
|-----------|---------|----------|-------------------------------------------------------------|
| rid       | integer | Yes      | Response ID; use 0 for initial sync, then last received rid |

#### curl:

```
# Initial sync  
curl "https://localhost:8080/api/v2/sync/maindata?rid=0" \  
  -b cookies.txt \  
  -H "Referer: https://localhost:8080"  
  
# Incremental sync (using last rid=42)  
curl "https://localhost:8080/api/v2/sync/maindata?rid=42" \  
  -b cookies.txt \  
  -H "Referer: https://localhost:8080"
```

#### JavaScript:

```
class QBitTorrentSync {  
  private rid = 0;  
  private intervalId: number | null = null;  
  
  startPolling(intervalMs = 2000) {  
    this.intervalId = window.setInterval(async () => {  
      const response = await fetch(  
        `https://localhost:8080/api/v2/sync/maindata?rid=${this.rid}`,  
        {  
          headers: { 'Referer': 'https://localhost:8080' },  
          credentials: 'include'  
        }  
      );  
      const data = await response.json();  
      this.rid = data.rid;  
      this.handleUpdate(data);  
    }, intervalMs);  
  }  
  
  private handleUpdate(data: MainDataResponse) {  
    if (data.torrents) {  
      // Full torrent data (initial sync)  
      console.log('Full sync:', Object.keys(data.torrents).length, 'torrents');  
    }  
    if (data.torrents_removed) {  
      console.log('Removed torrents:', data.torrents_removed.length, 'torrents');  
    }  
  }  
}
```

```

        console.log('Removed:', data.torrents_removed);
    }
    if (data.categories) {
        console.log('Categories updated:', data.categories);
    }
    if (data.server_state) {
        console.log('Global speeds:',
            `DL ${data.server_state.dl_info_speed}`,
            `UL ${data.server_state.up_info_speed}`
        );
    }
}

stop() {
    if (this.intervalId) clearInterval(this.intervalId);
}
}

```

### Response:

```

{
  "rid": 43,
  "full_update": false,
  "torrents": {
    "a1b2c3d4...": {
      "progress": 0.7534,
      "dlspeed": 5242880,
      "upspeed": 1048576,
      "eta": 1800,
      "state": "downloading"
    }
  },
  "torrents_removed": [],
  "categories": {},
  "tags": [],
  "trackers": {},
  "server_state": {
    "connection_status": "connected",
    "dht_nodes": 350,
    "dl_info_speed": 5242880,
    "dl_info_data": 10737418240,
    "up_info_speed": 1048576,
    "up_info_data": 2147483648,
    "queueing": true,
    "use_alt_speed_limits": false,
    "refresh_interval": 2000
  }
}

```



## 4.5 Application

### GET /api/v2/app/version

Get qBitTorrent version.

#### curl:

```
curl "https://localhost:8080/api/v2/app/version" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

**Response:** "v4.6.4" (plain text)

#### JavaScript:

```
const response = await fetch('https://localhost:8080/api/v2/app/version',
  headers: { 'Referer': 'https://localhost:8080' },
  credentials: 'include'
});
const version = await response.text();
console.log('qBitTorrent version:', version);
```

### GET /api/v2/app/webapiVersion

Get WebUI API version.

#### curl:

```
curl "https://localhost:8080/api/v2/app/webapiVersion" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

**Response:** "2.9.3" (plain text)

### GET /api/v2/app/preferences

Get application preferences.

#### curl:

```
curl "https://localhost:8080/api/v2/app/preferences" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

**Response:** Full preferences object (100+ fields).

### POST /api/v2/app/setPreferences

Set application preferences.

| Parameter | Type | Description |
|-----------|------|-------------|
|-----------|------|-------------|

|      |        |                                 |
|------|--------|---------------------------------|
| json | string | JSON-encoded preferences object |
|------|--------|---------------------------------|

**curl:**

```
curl -X POST "https://localhost:8080/api/v2/app/setPreferences" \  
  -b cookies.txt \  
  -H "Referer: https://localhost:8080" \  
  -d 'json={"max_active_downloads": 5, "max_active_torrents": 10}'
```

**GET /api/v2/app/buildInfo**

Get build information.

**curl:**

```
curl "https://localhost:8080/api/v2/app/buildInfo" \  
  -b cookies.txt \  
  -H "Referer: https://localhost:8080"
```

**Response:**

```
{  
  "qt": "6.6.2",  
  "libtorrent": "2.0.10.0",  
  "boost": "1.84.0",  
  "openssl": "3.2.1",  
  "zlib": "1.3.1"  
}
```

## 4.6 Transfer Info

**GET /api/v2/transfer/info**

Get global transfer information.

**curl:**

```
curl "https://localhost:8080/api/v2/transfer/info" \  
  -b cookies.txt \  
  -H "Referer: https://localhost:8080"
```

**JavaScript:**

```
const response = await fetch('https://localhost:8080/api/v2/transfer/info'  
  headers: { 'Referer': 'https://localhost:8080' },  
  credentials: 'include'  
});  
const info = await response.json();  
console.log(`Global DL: ${formatBytes(info.dl_info_speed)}/s`);  
console.log(`Global UL: ${formatBytes(info.up_info_speed)}/s`);  
console.log(`DHT Nodes: ${info.dht_nodes}`);
```

**Response:**

```
{  
  "connection_status": "connected",
```

```

    "dht_nodes": 350,
    "dl_info_speed": 5242880,
    "dl_info_data": 10737418240,
    "up_info_speed": 1048576,
    "up_info_data": 2147483648,
    "dl_rate_limit": 0,
    "up_rate_limit": 0,
    "queueing": true,
    "use_alt_speed_limits": false,
    "refresh_interval": 2000
}

```

## 4.7 qBitTorrent API Endpoint Summary

| Method | Endpoint                        | Description       |
|--------|---------------------------------|-------------------|
| POST   | /api/v2/auth/login              | Authenticate      |
| POST   | /api/v2/auth/logout             | End session       |
| POST   | /api/v2/torrents/add            | Add torrent(s)    |
| GET    | /api/v2/torrents/info           | List torrents     |
| POST   | /api/v2/torrents/delete         | Delete torrent(s) |
| POST   | /api/v2/torrents/pause          | Pause torrent(s)  |
| POST   | /api/v2/torrents/start          | Resume torrent(s) |
| POST   | /api/v2/torrents/recheck        | Force recheck     |
| POST   | /api/v2/torrents/reannounce     | Force reannounce  |
| GET    | /api/v2/torrents/categories     | List categories   |
| POST   | /api/v2/torrents/createCategory | Create category   |
| GET    | /api/v2/torrents/tags           | List tags         |
| POST   | /api/v2/torrents/addTags        | Add tags          |
| POST   | /api/v2/torrents/removeTags     | Remove tags       |
| GET    | /api/v2/sync/maindata           | Incremental sync  |
| GET    | /api/v2/app/version             | Get version       |
| GET    | /api/v2/app/webapiVersion       | Get API version   |
| GET    | /api/v2/app/preferences         | Get preferences   |
| POST   | /api/v2/app/setPreferences      | Set preferences   |
| GET    | /api/v2/app/buildInfo           | Get build info    |
| GET    | /api/v2/transfer/info           | Get transfer info |

---

## 5. TypeScript Type Definitions

### 5.1 Core Types

```

/**
 * Uniquely identifies a torrent by its 40-character SHA-1 infohash.
 */
type Infohash = string; // /^[a-f0-9]{40}$/

/**

```

```

    * Magnet URI as defined by BEP-0009.
    */
type MagnetUri = string; // /^magnet:\?xt=urn:btih:[a-fA-F0-9]{40}/

/**
 * Extension-defined priority levels for queue management.
 */
type Priority = 'low' | 'normal' | 'high' | 'critical';

/**
 * Server connection states.
 */
type ConnectionState = 'connected' | 'connecting' | 'disconnected' | 'erro

/**
 * Supported authentication methods.
 */
type AuthMethod = 'cookie' | 'api-key' | 'basic' | 'custom-header';

/**
 * Supported connection modes.
 */
type ConnectionMode = 'boba' | 'qbittorrent-direct';

/**
 * UI theme preference.
 */
type ThemePreference = 'system' | 'light' | 'dark';

/**
 * Queue item lifecycle status.
 */
type QueueStatus = 'pending' | 'retrying' | 'failed' | 'dead-letter';

/**
 * Torrent source type.
 */
type TorrentSource = 'magnet-link' | 'torrent-file';

```

## 5.2 Data Interfaces

```

/**
 * Represents a detected torrent with all parsed metadata.
 */
interface TorrentInfo {
    infohash: Infohash;
    displayName: string;
    magnetUri: MagnetUri;
    source: TorrentSource;
    trackers: string[];
    totalSize?: number;
    webSeeds?: string[];
    pageUrl: string;
}

```

```

    detectedAt: number;
    fileBlob?: Blob;
}

/**
 * Encrypted credential storage entry.
 */
interface EncryptedCredentials {
    /** Base64-encoded salt (16 bytes) */
    salt: string;

    /** Base64-encoded IV (12 bytes) + ciphertext + authTag (16 bytes) */
    ciphertext: string;

    /** Timestamp of encryption */
    encryptedAt: number;
}

/**
 * Server connection configuration.
 */
interface ServerConfig {
    mode: ConnectionMode;
    baseUrl: string;
    authMethod: AuthMethod;
    credentials: EncryptedCredentials;
    defaultCategory?: string;
    defaultTags?: string[];
    defaultSavePath?: string;
    timeout: number;
    healthCheckInterval: number;
}

/**
 * Complete extension configuration.
 */
interface ExtensionConfig {
    version: string;
    server: ServerConfig;
    ui: UIConfig;
    queue: QueueConfig;
    detection: DetectionConfig;
    rateLimit: RateLimitConfig;
}

interface UIConfig {
    theme: ThemePreference;
    badgeEnabled: boolean;
    notifications: NotificationPreferences;
}

interface NotificationPreferences {
    sendSuccess: boolean;
    sendFailed: boolean;
}

```

```

    batchComplete: boolean;
    queueRetry: boolean;
}

interface QueueConfig {
    maxSize: number;
    maxRetries: number;
    retryBaseDelayMs: number;
    retryMaxDelayMs: number;
}

interface DetectionConfig {
    scanDynamically: boolean;
    maxTorrentFileSize: number;
    enableTabGroupScan: boolean;
}

interface RateLimitConfig {
    requestsPerSecond: number;
    requestsPerMinute: number;
    interCallDelayMs: number;
}

```

### 5.3 Queue Types

```

interface QueueItem {
    id: string;
    torrent: TorrentInfo;
    status: QueueStatus;
    attemptCount: number;
    nextRetryAt: number;
    lastError?: string;
    lastHttpStatus?: number;
    queuedAt: number;
}

interface QueueSnapshot {
    items: QueueItem[];
    total: number;
    byStatus: Record<QueueStatus, number>;
}

```

### 5.4 API Response Types

```

interface ApiResponse<T> {
    success: boolean;
    data?: T;
    error?: ApiError;
    meta?: {
        requestId: string;
        timestamp: number;
        durationMs: number;
    };
};

```

```

}

interface ApiError {
  code: string;
  message: string;
  details?: Record<string, unknown>;
  httpStatus?: number;
}

interface ConnectionTestResult {
  success: boolean;
  serverUrl: string;
  mode: ConnectionMode;
  version?: string;
  latencyMs: number;
  error?: string;
}

```

## 5.5 qBitTorrent-Specific Types

```

interface QBitTorrentInfo {
  hash: string;
  name: string;
  magnet_uri: string;
  size: number;
  progress: number;
  dl_speed: number;
  up_speed: number;
  priority: number;
  num_seeds: number;
  num_complete: number;
  num_leechs: number;
  num_incomplete: number;
  ratio: number;
  eta: number;
  state: TorrentState;
  seq_dl: boolean;
  f_l_piece_prio: boolean;
  category: string;
  tags: string;
  super_seeding: boolean;
  force_start: boolean;
  save_path: string;
  added_on: number;
  completion_on: number;
  tracker: string;
  dl_limit: number;
  up_limit: number;
  downloaded: number;
  uploaded: number;
  downloaded_session: number;
  uploaded_session: number;
  amount_left: number;
}

```

```

    completed: number;
    max_ratio: number;
    max_seeding_time: number;
    seen_complete: number;
    last_activity: number;
    total_size: number;
    time_active: number;
    seeding_time: number;
    seeding_time_limit: number;
    availability: number;
    reannounce: number;
    stalled: boolean;
}

```

```

type TorrentState =
    | 'error'
    | 'missingFiles'
    | 'uploading'
    | 'pausedUP'
    | 'queuedUP'
    | 'stalledUP'
    | 'checkingUP'
    | 'forcedUP'
    | 'allocating'
    | 'downloading'
    | 'metaDL'
    | 'pausedDL'
    | 'queuedDL'
    | 'stalledDL'
    | 'checkingDL'
    | 'forcedDL'
    | 'checkingResumeData'
    | 'moving';

```

```

interface ServerState {
    connection_status: 'connected' | 'firewalled' | 'disconnected';
    dht_nodes: number;
    dl_info_speed: number;
    dl_info_data: number;
    up_info_speed: number;
    up_info_data: number;
    dl_rate_limit: number;
    up_rate_limit: number;
    queueing: boolean;
    use_alt_speed_limits: boolean;
    refresh_interval: number;
}

```

```

interface MainDataResponse {
    rid: number;
    full_update: boolean;
    torrents?: Record<string, Partial<QBitTorrentInfo>>;
    torrents_removed?: string[];
    categories?: Record<string, { name: string; savePath: string }>;
}

```



```

categories_removed?: string[];
tags?: string[];
tags_removed?: string[];
trackers?: Record<string, string[]>;
trackers_removed?: string[];
server_state?: ServerState;
}

```

## 5.6 Utility Types

```

/**
 * Deep partial type for partial config updates.
 */
type DeepPartial<T> = {
  [P in keyof T]?: T[P] extends object ? DeepPartial<T[P]> : T[P];
};

/**
 * Message handler function type.
 */
type MessageHandler<TPayload = unknown, TResponse = unknown> = (
  message: ExtensionMessage<TPayload>,
  sender: chrome.runtime.MessageSender
) => Promise<TResponse> | TResponse;

/**
 * Rate limit bucket state.
 */
interface RateLimitBucket {
  tokens: number;
  lastRefill: number;
  capacity: number;
}

```

---

## 6. Error Codes and Handling

### 6.1 Extension Error Codes

| Code           | HTTP Status | Description           | User Action                |
|----------------|-------------|-----------------------|----------------------------|
| E_UNKNOWN      | â€”         | Unexpected error      | Check logs, report issue   |
| E_NETWORK      | â€”         | Network unreachable   | Check connection, retry    |
| E_TIMEOUT      | 408         | Request timed out     | Retry, check server status |
| E_DNS          | â€”         | DNS resolution failed | Verify server URL          |
| E_CONN_REFUSED | â€”         | Connection refused    | Verify server is running   |
| E_AUTH         | 401         | Authentication failed | Check credentials          |

| Code                  | HTTP Status | Description                    | User Action                |
|-----------------------|-------------|--------------------------------|----------------------------|
| E_AUTH_EXPIRED        | 401         | Session expired                | Re-authenticate            |
| E_RATE_LIMIT          | 429         | Rate limited by server         | Wait and retry             |
| E_CLIENT_RATE_LIMIT   | â€”         | Client rate limit hit          | Reduce request frequency   |
| E_VALIDATION          | 400         | Invalid request data           | Check parameters           |
| E_SERVER              | 500         | Server error                   | Check server logs          |
| E_SERVICE_UNAVAILABLE | 503         | Server temporarily unavailable | Retry later                |
| E_TORRENT_INVALID     | 400         | Invalid torrent data           | Verify torrent file/URI    |
| E_TORRENT_DUPLICATE   | 409         | Torrent already exists         | Skip or force re-add       |
| E_STORAGE_FULL        | â€”         | Local storage quota exceeded   | Clear queue or storage     |
| E_STORAGE_CORRUPT     | â€”         | Stored data is corrupted       | Reset configuration        |
| E_CRYPT0              | â€”         | Encryption/decryption failed   | Re-enter credentials       |
| E_PERMISSION_DENIED   | 403         | Browser permission denied      | Grant required permissions |
| E_TAB_ACCESS          | â€”         | Cannot access tab              | Check tab permissions      |
| E_CORS                | â€”         | Cross-origin request blocked   | Use Boba proxy mode        |
| E_FILE_TOO_LARGE      | 413         | Torrent file exceeds limit     | Use magnet link instead    |
| E_UNSUPPORTED_BROWSER | â€”         | Browser feature unavailable    | Update browser             |

## 6.2 Error Response Format

All API errors from Boba follow this format:

```
{
  "error": "E_RATE_LIMIT",
  "message": "Too many requests. Please slow down.",
  "retryAfter": 30,
  "details": {
    "limit": 60,
    "window": 60,
    "remaining": 0
  }
}
```

## 6.3 Error Handling in Extension Code

```
async function handleApiCall<T>(
  operation: () => Promise<T>,
  context: string
```

```

): Promise<T> {
  try {
    return await operation();
  } catch (error) {
    if (error instanceof ExtensionError) {
      switch (error.code) {
        case 'E_AUTH':
        case 'E_AUTH_EXPIRED':
          await promptReauthentication();
          throw error;

        case 'E_RATE_LIMIT':
          const delay = error.retryAfter || 60;
          await scheduleRetry(delay * 1000);
          throw error;

        case 'E_NETWORK':
        case 'E_TIMEOUT':
          await queueForRetry(context);
          throw error;

        case 'E_TORRENT_DUPLICATE':
          // Silently skip duplicates
          return null as T;

        default:
          await logError(error, context);
          throw error;
      }
    }
    throw error;
  }
}

```

## 6.4 Retry Strategies

| Error Category      | Strategy                        | Max Retries | Backoff                            |
|---------------------|---------------------------------|-------------|------------------------------------|
| Network errors      | Exponential backoff             | 5           | 5s â†’ 10s â†’ 20s â†’ 40s â†’ 80s |
| Timeout             | Exponential backoff             | 3           | 2s â†’ 4s â†’ 8s                   |
| Rate limit (429)    | Honor Retry-After               | 10          | As specified by server             |
| Auth errors         | No automatic retry              | 0           | N/A â€” requires user action       |
| Server errors (5xx) | Exponential backoff with jitter | 5           | 5s â†’ 10s â†’ 20s â†’ 40s â†’ 80s |
| Validation errors   | No retry                        | 0           | N/A â€” fix input                  |

---

## 7. Rate Limiting

### 7.1 Client-Side Rate Limiter

The extension implements a token bucket rate limiter to prevent overwhelming the server.

```
class TokenBucketRateLimiter {
  private tokens: number;
  private lastRefill: number;

  constructor(
    private capacity: number,
    private refillRate: number, // tokens per second
    private initialTokens: number = capacity
  ) {
    this.tokens = initialTokens;
    this.lastRefill = Date.now();
  }

  /**
   * Attempt to consume tokens. Returns wait time in ms if not enough tokens
   */
  tryConsume(count: number = 1): { allowed: boolean; waitMs?: number } {
    this.refill();

    if (this.tokens >= count) {
      this.tokens -= count;
      return { allowed: true };
    }

    const deficit = count - this.tokens;
    const waitMs = Math.ceil((deficit / this.refillRate) * 1000);
    return { allowed: false, waitMs };
  }

  private refill(): void {
    const now = Date.now();
    const elapsedMs = now - this.lastRefill;
    const tokensToAdd = (elapsedMs / 1000) * this.refillRate;
    this.tokens = Math.min(this.capacity, this.tokens + tokensToAdd);
    this.lastRefill = now;
  }
}
```

### 7.2 Default Rate Limit Configuration

| Bucket           | Capacity | Refill Rate | Purpose            |
|------------------|----------|-------------|--------------------|
| Per-second       | 10       | 10/s        | Burst handling     |
| Per-minute       | 60       | 1/s         | Sustained rate     |
| Queue processing | 1        | 2/s         | Inter-call spacing |

## 7.3 Rate Limit Headers

The extension respects server-sent rate limit headers:

| Header                | Description        | Action                |
|-----------------------|--------------------|-----------------------|
| X-RateLimit-Limit     | Request quota      | Log for debugging     |
| X-RateLimit-Remaining | Remaining requests | Preemptive throttling |
| X-RateLimit-Reset     | Reset timestamp    | Calculate wait time   |
| Retry-After           | Seconds to wait    | Honor exactly         |

## 7.4 Queue Processing Rate Limit

When processing the offline queue:

```
const QUEUE_CONFIG = {
  maxConcurrent: 1,
  interCallDelayMs: 500,
  batchSize: 10,
  batchIntervalMs: 5000
};

async function processQueue(items: QueueItem[]): Promise<void> {
  for (const item of items) {
    await sendTorrent(item.torrent);
    await delay(QUEUE_CONFIG.interCallDelayMs);
  }
}
```

---

# 8. Version Compatibility Matrix

## 8.1 qBitTorrent Version Compatibility

| qBitTorrent WebAPI Status |         |             | Notes               |
|---------------------------|---------|-------------|---------------------|
| 4.4.x                     | 2.8.x   | Compatible  | Legacy support      |
| 4.5.x                     | 2.8.x   | Compatible  | Recommended minimum |
| 4.6.x                     | 2.9.x   | Compatible  | Primary target      |
| 5.0.x                     | 2.10.x  | Compatible  | Future-proof        |
| 5.1.x                     | 2.11.x  | Testing     | Under validation    |
| < 4.4.0                   | < 2.8.0 | Unsupported | Upgrade required    |

## 8.2 Browser Version Compatibility

| Browser | Minimum | Recommended | MV3 Support          | Tested |
|---------|---------|-------------|----------------------|--------|
| Chrome  | 88      | 120+        | Full                 | Yes    |
| Firefox | 109     | 121+        | Full (with polyfill) | Yes    |
| Opera   | 74      | 106+        | Full                 | Yes    |
| Yandex  | 21      | 24+         | Full                 | Yes    |

| Browser Minimum Recommended MV3 Support |    |      |         | Tested |
|-----------------------------------------|----|------|---------|--------|
| Edge                                    | 88 | 120+ | Full    | Yes    |
| Safari                                  | 16 | 17+  | Partial | No     |

### 8.3 Boba Server Compatibility

#### Boba Server Extension Status

|       |        |                       |
|-------|--------|-----------------------|
| 1.0.x | 1.0.x  | Compatible            |
| 1.1.x | 1.0.x  | Compatible (backward) |
| 1.2.x | 1.0.x  | Testing               |
| 2.0.x | 1.1.x+ | Planned               |

### 8.4 API Feature Availability

| Feature               | qBitTorrent | Direct | Boba | Proxy | Notes             |
|-----------------------|-------------|--------|------|-------|-------------------|
| Magnet send           | 4.4+        |        | All  |       | Core feature      |
| .torrent upload       | 4.4+        |        | All  |       | Core feature      |
| Categories            | 4.4+        |        | All  |       | Full CRUD         |
| Tags                  | 4.3.2+      |        | All  |       | Full CRUD         |
| Sequential download   | 4.4+        |        | All  |       | Add parameter     |
| First/last piece prio | 4.4+        |        | All  |       | Add parameter     |
| SSE progress          | N/A         |        | 1.1+ |       | Boba-only feature |
| Search aggregation    | N/A         |        | 1.2+ |       | Boba-only feature |
| Auto-discovery        | N/A         |        | 1.0+ |       | Boba-only feature |

---

*End of API Reference*